



RUSTYX

PROYECTO FIN DE GRADO

Juan Manuel Carrasco Benitez

Centro Educativo IES Albarregas

Documento Técnico Mínimos RustyX

Alumno: Juan Manuel Carrasco Benítez

Ciclo: ASIR Administración de Sistemas Informáticos en Red

Índice

1. Resumen	4
2. Introducción	4
2.1. Alcance de la Infraestructura	4
2.2. Metodología	5
3. Descripción de la Empresa	5
3.1. Nombre y Sector	5
3.2. Tamaño y Estructura	5
3.3. Necesidades Tecnológicas	5
4. Infraestructura Física y Virtual	6
4.1. Plataforma de Virtualización	6
4.2. Requisitos de Hardware	6
4.3. Requisitos de Software	6
5. Diseño de Red	6
5.1. Arquitectura de Tres Capas	6
5.2. Direccionamiento IP y Segmentación	7
5.3. Puertos y Protocolos	9
5.4. Políticas de Firewall (iptables)	9
6. Base de Datos	9
6.1. Descripción General	9
6.2. Modelo Entidad-Relación	9
6.3. Descripción de Tablas	10
6.4. Acceso y Seguridad de la Base de Datos	13
7. Infraestructura de Servicios de Red (Windows Server)	13
7.1. Descripción General	13
7.2. Servicio DHCP	13
7.3. Servicio DNS	13
7.4. Servicio FTP / TFTP	13
8. Servicios de Directorio — Active Directory	14
8.1. Descripción y Justificación	14
8.2. Nombre del Dominio	14
8.3. Estructura de Unidades Organizativas (OUs)	14
8.4. Grupos de Seguridad	14
8.5. Directiva de Nombres y Contraseñas	15
8.6. Perfiles Móviles	15
8.7. Políticas de Grupo (GPO)	16

8.8. Script de Automatización de Usuarios (PowerShell)	17
9. Infraestructura de Servicios Web (Docker)	19
9.1. Descripción General	19
9.2. Balanceador de Carga	19
9.3. Servidores Web Backend	19
9.5. Procedimientos Operativos	20
10. Herramientas de Gestión	20
10.1. Sistema de Tickets de Incidencias (RustyX Tickets)	20
Descripción	20
Archivos del sistema	20
Flujo de trabajo	21
Descripción de funciones principales	21
10.2. Panel de Monitorización RustyX	22
Descripción y contexto	22
Arquitectura del panel	22
Script Bash — recopilar_datos.sh	23
Interfaz gráfica Python — panel_rustyx.py	24
Mejoras propuestas para el panel	24
11. Seguridad	24
11.1. Principios de Seguridad Aplicados	24
11.2. Seguridad en la Capa de Red	25
11.3. Seguridad en el Dominio Windows	25
11.4. Seguridad en la Capa Web	25
11.5. Seguridad en la Base de Datos	25
12. Alta Disponibilidad y Tolerancia a Fallos	26
12.1. Puntos Críticos del Sistema	26
12.2. Mecanismos de Redundancia Implementados	26
12.3. Escenarios de Fallo y Respuesta	26
12.4. Propuestas de Mejora de Alta Disponibilidad	26
13. Resolución de Problemas	27
13.1. Problemas Habituales	27
13.2. Comandos de Diagnóstico Básico	27
14. Glosario de Términos	28
15. Conclusiones y Propuestas de Mejora	28
15.1. Conclusiones	28
15.2. Propuestas de Mejora	29
16. Bibliografía	29
Parte extendida (Prácticas)	30
1. Documentación Técnica	30
1.1. Descripción General	30
1.2. Estructura General del Proyecto	30
1.3. Conexión con la Base de Datos	30
1.4. Catálogo de Videojuegos	30
1.5. Sistema de Filtros	31

1.6. Paginación de Juegos	31
1.7. Cambio de Formato de los Juegos	32
1.8. Página de Detalle del Juego	32
1.9. Vídeos de YouTube Embebidos	32
1.10. Valoraciones	33
1.11. Comentarios	33
1.12. Listas Personales	33
1.13. Panel de Administración	33
1.14. Exportación CSV	34
1.15. Seguridad y Buenas Prácticas	35
2. Funcionalidad de la Página Web	35

1. Resumen

RustyX es una empresa del sector tecnológico especializada en el desarrollo de videojuegos. En este documento se cubre desde la capa más interna del sistema (base de datos) hasta la más externa (servicios de red y herramientas de gestión).

La infraestructura se articula en tres grandes bloques:

- Un servidor Windows Server que centraliza los servicios de directorio (Active Directory), DHCP, DNS y FTP/TFTP, gestionando todos los usuarios y políticas de seguridad corporativas.
- Un clúster web basado en contenedores Docker con balanceo de carga Nginx, dos servidores web backend redundantes y una base de datos MariaDB aislada, que da servicio al catálogo online de videojuegos de RustyX.
- Herramientas de gestión propias: un sistema de tickets de incidencias desarrollado en Python y un panel de monitorización en tiempo real (Bash + Python) para supervisar los contenedores Docker.

Objetivos del proyecto:

- Diseñar y desplegar una infraestructura segura y con alta disponibilidad para RustyX.
- Centralizar la gestión de usuarios mediante Active Directory con GPOs de seguridad.
- Implementar un catálogo web online con arquitectura LEMP de tres capas en Docker. (Posible parte Marzo - Junio)
- Desarrollar herramientas internas de gestión de incidencias y monitorización.

2. Introducción

El proyecto tiene como finalidad diseñar, implementar y documentar completamente la infraestructura tecnológica de RustyX, una empresa del sector del videojuego. La documentación está ordenada de lo más interno a lo más externo: comienza por la capa de datos (base de datos relacional), asciende por los servicios web y de red, y termina con las herramientas de gestión y monitorización que los técnicos utilizan en su día a día.

2.1. Alcance de la Infraestructura

El alcance del proyecto abarca:

- Segmentación y diseño de red con múltiples subredes aisladas (DMZ, backend, datos).
- Infraestructura de servicios de red basada en Windows Server: DHCP, DNS, FTP/TFTP y Active Directory.
- Infraestructura de servicios web basada en contenedores Docker: balanceador Nginx, dos servidores web Nginx/PHP-FPM y base de datos MariaDB.
- Base de datos relacional para el catálogo online de videojuegos.
- Herramientas de gestión propias: sistema de tickets (Python) y panel de monitorización (Bash + Python).

2.2. Metodología

La infraestructura se despliega íntegramente sobre virtualización (Docker y VirtualBox), lo que permite reproducir el entorno completo en un único equipo físico. Se prioriza la seguridad mediante el principio de mínimo privilegio, la segmentación estricta de red y las políticas de grupo aplicadas en el dominio Windows.

3. Descripción de la Empresa

3.1. Nombre y Sector

RustyX es una empresa del sector tecnológico dedicada al desarrollo de videojuegos. Además de su actividad productiva, opera un catálogo online que permite a sus clientes explorar y valorar los títulos disponibles, y que internamente sirve como herramienta de análisis de mercado para identificar los videojuegos mejor valorados y orientar su estrategia comercial.

3.2. Tamaño y Estructura

La organización cuenta con aproximadamente 20-25 empleados distribuidos en los siguientes departamentos:

Departamento	Subdepartamentos / Roles
Administración	Administración General, Finanzas, RRHH
Desarrollo	Backend, Frontend, Testers

Infraestructura	Administradores de sistemas
Dirección	Dirección general
Marketing	Marketing y comunicación
Atención al Cliente	Soporte y atención

3.3. Necesidades Tecnológicas

Para cubrir sus operaciones se requiere:

- Gestión centralizada de usuarios y equipos mediante un dominio Windows.
- Control de acceso diferenciado por departamento mediante grupos y políticas de seguridad.
- Aplicación de políticas de seguridad homogéneas en todos los equipos del dominio.
- Perfiles móviles para directivos y administradores.
- Automatización de la creación de cuentas de usuario.
- Un catálogo web online con alta disponibilidad.
- Herramientas internas de gestión de incidencias y monitorización de la infraestructura.

4. Infraestructura Física y Virtual

4.1. Plataforma de Virtualización

Toda la infraestructura se despliega sobre un único host físico combinando Docker (para los servicios web) y VirtualBox (para el servidor Windows). Este enfoque permite reproducir el entorno completo sin hardware dedicado, dimensionado para soportar aproximadamente 1.000 peticiones por segundo en los servicios web.

4.2. Requisitos de Hardware

Componente	vCP U	RAM	Disco	Notas
Windows Server (AD, DHCP, DNS, FTP)	2	4 GB	40 GB	Controlador de dominio
Balanceador Nginx (Docker)	0,5	256 MB	100 MB	Punto de entrada web
Servidor Web 1 (Docker)	1	512 MB	500 MB	Backend Nginx/PHP-FPM
Servidor Web 2 (Docker)	1	512 MB	500 MB	Backend Nginx/PHP-FPM
MariaDB (Docker)	1	1 GB	5 GB	Base de datos aislada

4.3. Requisitos de Software

Capa	Software	Versión / Distribución
Sistema Operativo (Windows)	Windows Server	2019 / 2022
Sistema Operativo (Linux)	Debian	12 Bookworm
Servidor Web	Nginx + PHP-FPM	Última estable
Base de Datos	MariaDB Server	10.x
Contenedores	Docker + iptables	Última estable

5. Diseño de Red

5.1. Arquitectura de Tres Capas

La arquitectura de red sigue un modelo de tres capas, separando el tráfico público del tráfico interno y aislando completamente la capa de datos. Esto implementa el principio de defensa en profundidad: cada capa solo puede comunicarse con la adyacente y ningún componente expone directamente la base de datos a internet.

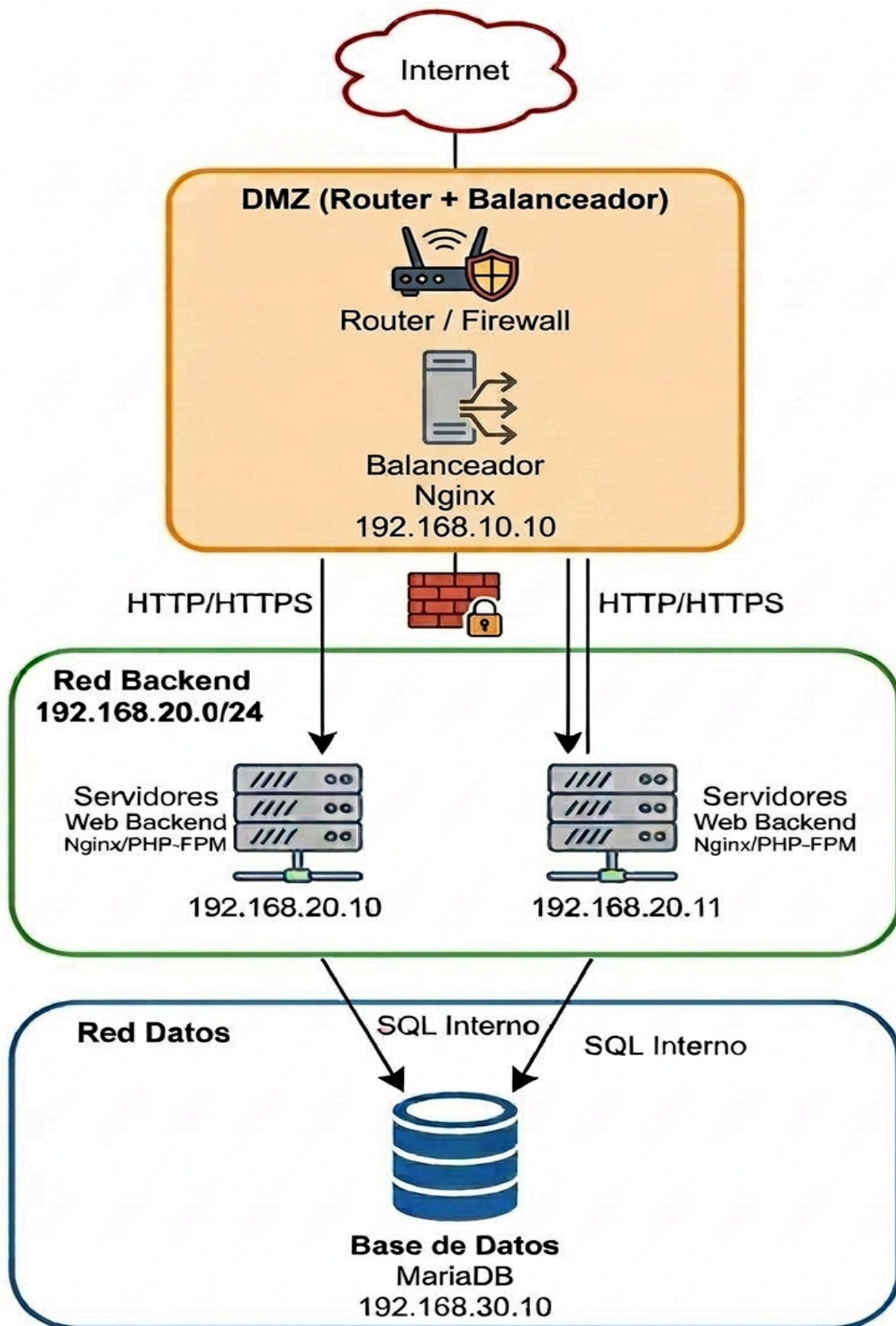
Flujo de datos:

1. La petición HTTP/HTTPS del cliente llega al balanceador en la DMZ.
2. El balanceador distribuye la petición (Round-Robin) a uno de los backends disponibles.
3. El backend procesa PHP y consulta MariaDB vía SQL interno.
4. La respuesta retorna por la misma ruta hasta el cliente.

5.2. Direccionamiento IP y Segmentación

Componente	Red	Dirección IP	Gateway	Función
Router / Firewall	192.168.10.0/24	192.168.10.1	—	Perimetral / NAT
Balanceador Nginx	DMZ — 192.168.10.0/24	192.168.10.10	192.168.10.1	Entrada pública HTTP/HTTPS
Servidor Web Backend 1	192.168.20.0/24	192.168.20.10	Ninguno	Aplicación PHP redundante
Servidor Web Backend 2	192.168.20.0/24	192.168.20.11	Ninguno	Aplicación PHP redundante

MariaDB	192.168.30.0/24	192.168.30.1 0	Ninguno	Almacenamiento de datos aislado
---------	-----------------	-------------------	---------	------------------------------------



5.3. Puertos y Protocolos

Origen	Destino	Puerto	Protocolo	Descripción
Internet / Clientes	Balanceador (192.168.10.10)	80 / 443	TCP	HTTP / HTTPS público
Balanceador	Web 1 (192.168.20.10)	80 / 443	TCP	Tráfico balanceado
Balanceador	Web 2 (192.168.20.11)	80 / 443	TCP	Tráfico balanceado
Web 1 / Web 2	MariaDB (192.168.30.10)	3306	TCP	Consultas SQL internas
Administrador	Windows Server	3389	TCP	Escritorio Remoto (RDP)

5.4. Políticas de Firewall (iptables)

Las reglas de firewall se aplican por capa, siguiendo el principio de mínimo privilegio:

- Balanceador: acepta tráfico entrante únicamente en los puertos 80 y 443. Todo el tráfico restante se descarta.
- Servidores Backend: aceptan conexiones únicamente desde la IP del balanceador (192.168.10.10) en los puertos 80/443. Rechazan cualquier acceso directo desde internet.
- Base de Datos: acepta conexiones al puerto 3306 exclusivamente desde la subred backend (192.168.20.0/24). El acceso desde cualquier otra red se bloquea a nivel de firewall.

6. Base de Datos

6.1. Descripción General

La capa de base de datos es el componente más interno de la infraestructura. Se implementa con MariaDB sobre un contenedor Docker en la red de datos aislada (192.168.30.0/24), accesible únicamente desde los servidores web backend. Almacena toda la información del catálogo online de RustyX: usuarios, videojuegos, valoraciones, comentarios y listas personalizadas.

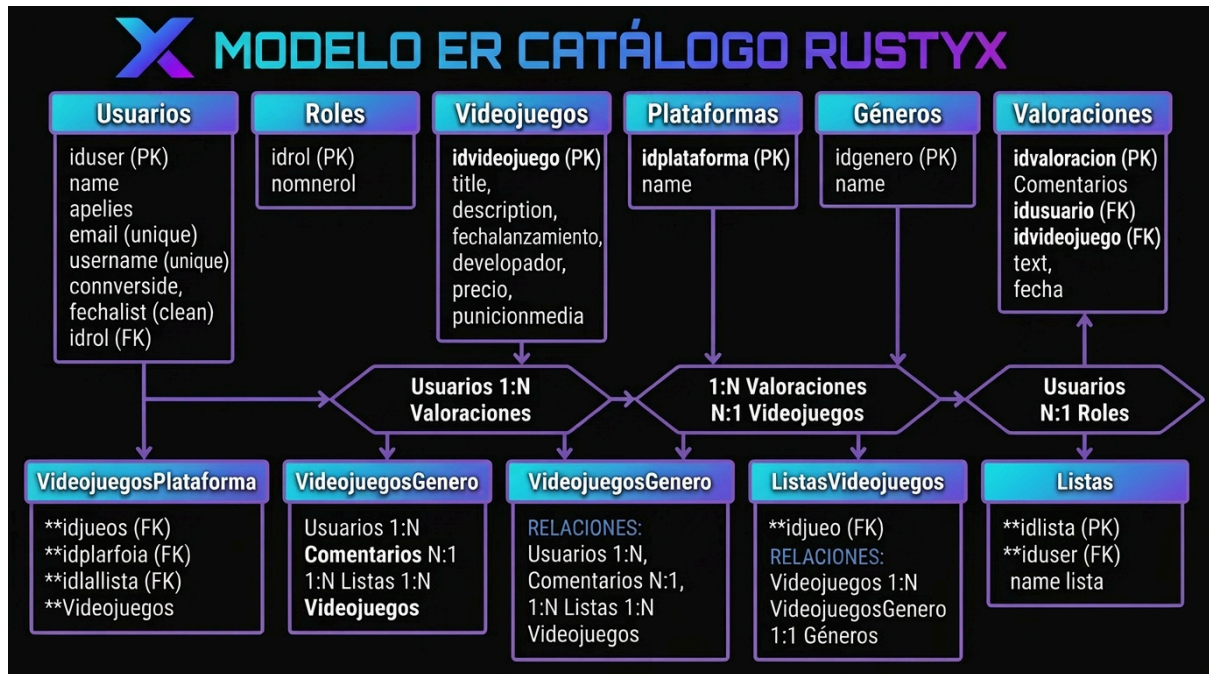
6.2. Modelo Entidad-Relación

El modelo relacional del catálogo se compone de 11 entidades. Las relaciones principales son:

Usuarios (1,N) —< Valoraciones >— (N,1) Videojuegos

Usuarios (1,N) —< Comentarios >— (N,1) Videojuegos

Usuarios (1,N) —< Listas >———— (1,N) Videojuegos [vía Lista_Videojuego]
 Videojuegos (1,N) —< Videojuego_Plataforma >— (N,1) Plataformas
 Videojuegos (1,N) —< Videojuego_Genero >———— (1,1) Géneros
 Usuarios (N,1) — Roles



6.3. Descripción de Tablas

Tabla usuarios: Almacena los datos de los usuarios registrados en el catálogo, incluyendo credenciales y nivel de permisos mediante el campo id_rol.

Campo	Tipo	PK	FK	Notas
id_usuario	INT AUTO_INCREMENT	✓		
nombre	VARCHAR(50)			
apellidos	VARCHAR(80)			
email	VARCHAR(80)			UNIQUE
username	VARCHAR(50)			UNIQUE
contraseña	VARCHAR(255)			Hash bcrypt recomendado
fecha_registro	DATE			
id_rol	INT		✓	→ roles

Tabla roles: Diferencia los tipos de usuarios del sistema: admin, desarrollador, tester, usuario normal, entre otros.

Campo	Tipo	PK
id_rol	INT AUTO_INCREMENT	✓
nombre_rol	VARCHAR(40)	

Tabla videojuegos: Contenido principal del catálogo. Cada videojuego incluye su descripción, precio y puntuación media calculada a partir de las valoraciones de los usuarios.

Campo	Tipo	PK	Notas
id_juego	INT AUTO_INCREMENT	✓	
titulo	VARCHAR(100)		
descripcion	TEXT		
fecha_lanzamiento	DATE		
desarrollador	VARCHAR(80)		
precio	DECIMAL(5,2)		
puntuacion_media	DECIMAL(3,2)		Calculada automáticamente

Tabla plataformas: Catálogo de plataformas disponibles: PC, PS5, Xbox Series X, Nintendo Switch, etc

Campo	Tipo	PK
id_plataforma	INT AUTO_INCREMENT	✓
nombre	VARCHAR(50)	

Tabla generos: Catálogo de géneros: Acción, Aventura, RPG, Shooter, etc.

Campo	Tipo	PK
id_genero	INT AUTO_INCREMENT	✓
nombre	VARCHAR(50)	

Tabla valoraciones: Puntuaciones del 1 al 10 asignadas por los usuarios a cada videojuego. Un usuario solo puede valorar una vez cada juego (restricción UNIQUE sobre id_usuario + id_juego).

Campo	Tipo	PK	FK	Notas
id_valoracion	INT AUTO_INCREMENT	✓		
id_usuario	INT		✓	→ usuarios
id_juego	INT		✓	→ videojuegos
puntuacion	TINYINT			Valor entre 1 y 10
fecha	DATE			

Tabla comentarios: Comentarios escritos por los usuarios sobre los videojuegos del catálogo.

Campo	Tipo	PK	FK
id_comentario	INT AUTO_INCREMENT	✓	
id_usuario	INT		✓
id_juego	INT		✓
texto	TEXT		
fecha	DATETIME		

Tabla lista: Permite a cada usuario crear listas personalizadas de juegos (tipo "lista de deseos", "jugados", etc.).

Campo	Tipo	PK	FK
id_lista	INT AUTO_INCREMENT	✓	
id_usuario	INT		✓
nombre_lista	VARCHAR(100)		

Tablas intermedias (relaciones N:M)

Tabla	Campos (FK)
videojuego_plataforma	id_juego (→ videojuegos), id_plataforma (→ plataformas)
videojuego_genero	id_juego (→ videojuegos), id_genero (→ géneros)
lista_videojuego	id_lista (→ lista), id_videojuego (→ videojuegos)

6.4. Acceso y Seguridad de la Base de Datos

El usuario de aplicación `rustyx_user` tiene permisos CRUD (SELECT, INSERT, UPDATE, DELETE) únicamente sobre la base de datos `rustyx_db`. Las conexiones remotas están restringidas al rango 192.168.20.0/24 (red backend). El puerto 3306 está bloqueado a nivel de firewall para cualquier otro origen.

7. Infraestructura de Servicios de Red (Windows Server)

7.1. Descripción General

El servidor Windows Server actúa como eje central de la infraestructura administrativa de RustyX. Centraliza los servicios que permiten la operación de toda la red corporativa: DHCP, DNS, FTP/TFTP y Active Directory. Su correcto funcionamiento es crítico para que los equipos del dominio puedan recibir configuración de red, resolver nombres y autenticar usuarios.

7.2. Servicio DHCP

El servicio DHCP automatiza la asignación de direcciones IP a todos los dispositivos de la red corporativa, eliminando la necesidad de configuración manual en cada equipo. Trabaja en estrecha colaboración con el servicio DNS: cuando un equipo recibe su IP, el servidor DNS puede actualizar su registro de forma dinámica.

- Asignación automática de IPs a los equipos del dominio RustyX.
- Integración con DNS para actualización dinámica de registros.
- Cualquier cambio de direccionamiento se gestiona desde un único punto sin reconfigurar cada dispositivo individualmente.

7.3. Servicio DNS

El DNS traduce los nombres del dominio (como `rustyx.local` o los nombres de los servidores) a sus direcciones IP correspondientes. Es imprescindible para el correcto funcionamiento de Active Directory, ya que los clientes localizan el controlador de dominio a través de registros DNS SRV.

7.4. Servicio FTP / TFTP

El servidor proporciona servicios de transferencia de archivos mediante FTP y TFTP. Son especialmente útiles para:

- Despliegue de actualizaciones y scripts de mantenimiento entre servidores.
- Sincronización de contenidos web entre componentes de la infraestructura.
- Transferencia de archivos de configuración entre los distintos nodos del sistema.

8. Servicios de Directorio — Active Directory

8.1. Descripción y Justificación

Active Directory (AD) es el componente más importante de la infraestructura de servicios, ya que proporciona una gestión centralizada de usuarios, grupos, permisos y políticas de seguridad. Permite administrar de forma organizada el acceso a todos los recursos de la red desde un único punto, simplificando la administración y reduciendo los riesgos de seguridad asociados a la gestión dispersa de cuentas.

8.2. Nombre del Dominio

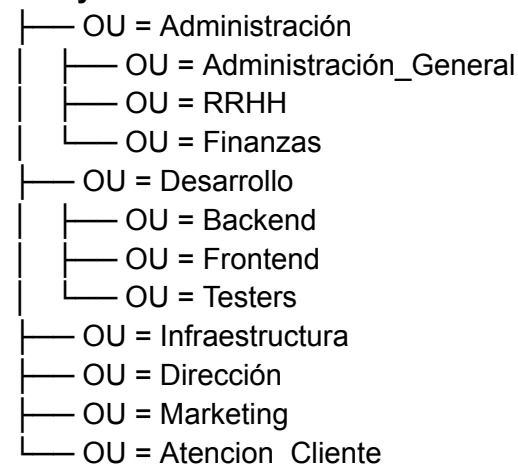
El dominio corporativo está configurado con el nombre RustyX.com

Cumple los estándares de nomenclatura internos e identifica claramente a la empresa.

8.3. Estructura de Unidades Organizativas (OUs)

La estructura de OUs refleja la organización real de RustyX y permite la aplicación selectiva de GPOs, la delegación administrativa por departamento y la gestión diferenciada de usuarios y equipos según su función:

RustyX.com



8.4. Grupos de Seguridad

Cada unidad organizativa tiene asignado su propio grupo de seguridad. El ámbito Global permite asignar permisos basados en el departamento y es plenamente compatible con la estructura multi-OU:

Grupo	Ámbito	Tipo
Administradores_RustyX	Global	Seguridad
RRHH_RustyX	Global	Seguridad
Finanzas_RustyX	Global	Seguridad
Backend_RustyX	Global	Seguridad
Frontend_RustyX	Global	Seguridad
Testers_RustyX	Global	Seguridad
Infraestructura_RustyX	Global	Seguridad
Directivos_RustyX	Global	Seguridad
Marketing_RustyX	Global	Seguridad
Atencion_Cliente_RustyX	Global	Seguridad
Jefes_Departamentos	Global	Seguridad

8.5. Directiva de Nombres y Contraseñas

Estándar de nombres de usuario Los usernames se generan con el patrón: 2 primeras letras del nombre + 2 primeras letras del apellido + número de orden (01).

Nombre completo	Username generado
Juan Pérez	jupe01
Laura Gómez	lago01

Estándar de contraseña inicial La contraseña sigue el patrón: Inicial del nombre (mayúscula) + Apellido completo + "1_". El usuario debe cambiarla en el primer inicio de sesión.

Nombre completo	Contraseña inicial
Juan Pérez	JPerez1_
Laura Gómez Ruiz	LGomez1_

Modificación de política de contraseñas Se ha ajustado la longitud mínima a 4 caracteres en la Default Domain Policy para facilitar las pruebas durante la fase de desarrollo.

8.6. Perfiles Móviles

Se configura una carpeta compartida en el servidor (C:\Perfiles\PerfilesMoviles, compartida como perfiles\$) para almacenar los perfiles móviles de directivos y administradores. Al

iniciar sesión en cualquier equipo del dominio, el perfil del usuario se carga desde el servidor, garantizando un entorno de escritorio consistente en cualquier máquina. El problema inicial de permisos en la carpeta fue identificado y corregido correctamente.

8.7. Políticas de Grupo (GPO)

Se han implementado 8 GPOs: 4 obligatorias y 4 adicionales diseñadas para reforzar la seguridad corporativa.

GPO 1 — Bloqueo del Panel de Control y CMD

Configuración	Ruta GPO
Prohibir acceso al Panel de Control	Config. Usuario → Plantillas Admin. → Panel de Control → Prohibir el acceso al Panel de control
Bloquear ejecución de cmd.exe	Config. Usuario → Plantillas Admin. → Sistema → Impedir el acceso al símbolo del sistema

Justificación: evita que usuarios sin conocimientos técnicos realicen cambios en la configuración del sistema o ejecuten comandos potencialmente dañinos.

GPO 2 — Fondo de Escritorio Corporativo y Contraseñas

Configuración	Ruta GPO
Fondo de pantalla corporativo	Config. Usuario → Plantillas Admin. → Active Desktop → Papel tapiz del escritorio
Historial de contraseñas (24)	Config. Equipo → Directivas → Config. Windows → Config. Seguridad → Directivas de cuenta → Directiva de contraseñas → Aplicar historial de contraseñas

Justificación: unifica la imagen corporativa en todos los equipos y refuerza la política de contraseñas impidiendo la reutilización de las últimas 24 contraseñas.

GPO 3 — Restricción Horaria

Horario permitido de inicio de sesión: 07:00 a 18:00. Fuera de ese horario, las sesiones activas se cierran automáticamente.

Ruta: Config. Usuario → Plantillas Admin. → Componentes de Windows → Opciones de inicio de sesión de Windows → Establecer acción que se debe realizar cuando finaliza el horario de inicio de sesión → Cerrar sesión.

Justificación: permite controlar el horario digital de los trabajadores y limitar el acceso fuera del horario laboral.

GPO 4 — Bloqueo de una Web Concreta

Bloquea el acceso a redes sociales u otras páginas no autorizadas asignándolas a la zona de sitios restringidos de Internet Explorer.

Ruta: Config. Usuario → Plantillas Admin. → Componentes de Windows → Internet Explorer → Panel de control de Internet → Página Seguridad → Asignar sitios a zonas.

Justificación: protege al dominio evitando el acceso a páginas no deseadas que puedan suponer un riesgo de seguridad o una pérdida de productividad.

GPO 5 — Desactivar Puertos USB (creada por el alumno)

Se deniegan los permisos de lectura, escritura y ejecución sobre todos los discos extraíbles.

Ruta: Config. Equipo → Plantillas Admin. → Sistema → Acceso al almacenamiento extraíble → Discos extraíbles: denegar acceso de lectura / escritura / ejecución (tres directivas habilitadas).

Justificación: impide la entrada o salida de datos no autorizada mediante dispositivos USB, mejorando la seguridad física de la información corporativa.

GPO 6 — Bloquear Instalación de Software

Desactiva Windows Installer para todos los usuarios (opción "Siempre").

Ruta: Config. Equipo → Plantillas Admin. → Componentes de Windows → Windows Installer → Desactivar Windows Installer → Siempre.

Justificación: evita que cualquier usuario instale software no autorizado que pueda comprometer la seguridad o la estabilidad del sistema.

GPO 7 — Bloquear el Administrador de Tareas

Elimina el acceso al Administrador de Tareas (taskmgr.exe).

Ruta: Config. Usuario → Plantillas Admin. → Sistema → Opciones de Ctrl+Alt+Supr → Quitar Administrador de tareas.

Justificación: evita que los usuarios cierren procesos importantes del sistema o intenten saltarse otras restricciones de seguridad.

GPO 8 — Quitar "Ejecutar" del Menú Inicio

Elimina la opción "Ejecutar" del menú Inicio y del Administrador de Tareas.

Ruta: Config. Usuario → Plantillas Admin. → Menú Inicio y barra de tareas → Quitar el menú Ejecutar del menú Inicio.

Justificación: impide que los usuarios lancen comandos avanzados, scripts o rutas UNC directamente desde el sistema, reduciendo la superficie de ataque.

8.8. Script de Automatización de Usuarios (PowerShell)

Se ha desarrollado un script PowerShell que automatiza la creación de cuentas en Active Directory respetando los estándares corporativos de nomenclatura y contraseñas. El script realiza las siguientes operaciones:

1. Solicita por consola el nombre, apellido, ruta de la OU y grupo destino.
2. Limpia el texto de acentos y espacios para generar nombres de usuario válidos en AD.
3. Valida que nombre y apellido tengan al menos 2 caracteres.

4. Verifica que el username generado no exista previamente en Active Directory.
5. Genera el username y la contraseña inicial según los estándares definidos.
6. Crea el usuario con la flag "Cambiar contraseña en el próximo inicio de sesión" activada.
7. Añade el usuario al grupo de seguridad indicado.
8. Muestra un resumen completo con usuario, contraseña inicial, OU y grupo asignado.

```
powershell
Import-Module ActiveDirectory

function Remove-Accents {
    param([string]$text)
    $text = $text.Trim()
    $normalized = $text.Normalize([Text.NormalizationForm]::FormD)
    $result = $normalized -replace '\p{M}', ''
    return $result
}

$nombre = Read-Host "Nombre"
$apellido = Read-Host "Apellido"
$ou = Read-Host "Ruta OU (Ej:
OU=Backend,OU=Desarrollo,DC=rustyx,DC=local)"
$grupo = Read-Host "Grupo al que añadir el usuario"

$nombreLimpio = Remove-Accents $nombre
$apellidoLimpio = Remove-Accents $apellido

# Username: 2 letras nombre + 2 letras apellido + "01"
$user = ($nombreLimpio.Substring(0,2) + $apellidoLimpio.Substring(0,2) +
"01").ToLower()
# Contraseña: Inicial mayúscula + apellido completo + "1_"
$passText = $nombreLimpio.Substring(0,1).ToUpper() + $apellidoLimpio +
"1_"
$pass = ConvertTo-SecureString $passText -AsPlainText -Force

New-ADUser
    -Name "$nombre $apellido"
    -SamAccountName $user
    -UserPrincipalName "$user@rustyx.local"
    -GivenName $nombre
    -Surname $apellido
    -AccountPassword $pass
    -ChangePasswordAtLogon $true
    -Enabled $true
    -Path $ou
Add-ADGroupMember -Identity $grupo -Members $user
```

9. Infraestructura de Servicios Web (Docker)

9.1. Descripción General

La infraestructura web de RustyX está desplegada en contenedores Docker y sigue una arquitectura de tres capas estrictamente segmentada: balanceador de carga en la DMZ, capa de servidores web backend redundantes y capa de base de datos aislada. Esta separación garantiza alta disponibilidad, seguridad y la posibilidad de escalar horizontalmente añadiendo más nodos backend sin interrumpir el servicio.

9.2. Balanceador de Carga

El balanceador es el único punto de entrada de la infraestructura web. Se despliega en la DMZ y distribuye todas las peticiones HTTP/HTTPS entre los servidores backend mediante Round-Robin.

Parámetro	Valor
Dirección IP	192.168.10.10
Red	DMZ — 192.168.10.0/24
Software	Nginx (proxy inverso)
Algoritmo de balanceo	Round-Robin equitativo
Puertos expuestos	80 (HTTP) y 443 (HTTPS)
Health checks	Pasivos: errores 5xx / timeout marcan el backend como no disponible

Las comprobaciones de estado pasivas garantizan que, si un backend falla, el tráfico se redirige automáticamente al nodo restante sin intervención manual.

9.3. Servidores Web Backend

Se despliegan dos servidores web con configuraciones idénticas. Ambos ejecutan Nginx como servidor HTTP con PHP-FPM para el procesamiento de PHP, y alojan la aplicación RustyX en `/var/www/rustyx/`.

Parámetro	Servidor Web 1	Servidor Web 2
Dirección IP	192.168.20.10	192.168.20.11
Red	192.168.20.0/24	192.168.20.0/24
Software	Nginx + PHP-FPM	Nginx + PHP-FPM
Contenido	<code>/var/www/rustyx/</code>	<code>/var/www/rustyx/</code>
Permisos dirs/archivos	755 / 644 (owner: www-data)	755 / 644 (owner: www-data)

La redundancia garantiza que si uno de los servidores cae, el sistema continúa funcionando al 50% de capacidad a través del nodo disponible. Los logs se encuentran en `/var/log/nginx/access.log` y `/var/log/nginx/error.log`. El estado de PHP-FPM se consulta con `journalctl -u php8.1-fpm`.

9.5. Procedimientos Operativos

Arranque y parada de servicios

El orden es crítico para evitar errores de conexión entre capas:

Operación	Orden
Arranque	1. MariaDB → 2. Servidores Backend → 3. Balanceador
Parada	1. Balanceador → 2. Servidores Backend → 3. MariaDB

Alta / Baja de un servidor web

- **Alta:** clonar VM/contenedor → sincronizar /var/www/rustyx/ → añadir IP al bloque upstream del balanceador → ejecutar `nginx -s reload`.
- **Baja:** quitar IP del bloque upstream → esperar a que drenen las conexiones activas → parar los servicios.

10. Herramientas de Gestión

10.1. Sistema de Tickets de Incidencias (RustyX Tickets)

Descripción

RustyX Tickets es una aplicación de consola desarrollada en Python que centraliza la gestión de incidencias internas de la empresa. Permite registrar y consultar problemas relacionados con aplicaciones web, bases de datos, red y servidores, clasificados por tipo, prioridad y estado. Todos los usuarios comparten el mismo menú principal una vez autenticados.

Archivos del sistema

Archivo	Formato	Descripción
incidencias.csv	CSV (delimitador ;)	Almacena todas las incidencias registradas
usuarios.csv	CSV (delimitador ;)	Credenciales y roles de los empleados autorizados

Estructura de incidencias.csv:

Campo	Tipo	Descripción
ID	Integer	Identificador único correlativo, generado automáticamente
Fecha	DATE	Fecha de registro (YYYY-MM-DD)
Hora	TIME	Hora exacta de registro (HH:MM:SS)
Usuario	String	Usuario que creó la incidencia
Tipo	Enum	WEB / BD / RED / SERVIDOR
Prioridad	Enum	BAJA / MEDIA / ALTA
Descripcion	Text	Detalle del problema reportado
Estado	Enum	ABIERTA / EN CURSO / CERRADA

Estructura de usuarios.csv:

Campo	Descripción
usuario	Nombre de usuario único para el login
password	Contraseña de acceso (en producción debe estar cifrada)
nombre	Rol: administrador, soporte, tecnico

Flujo de trabajo

INICIO → verificar_login(usuario, password)
 ↳ Lee usuarios.csv con DictReader (delimiter=';')
 ↳ Compara usuario y password (con strip())

MENÚ PRINCIPAL:

1. Crear Incidencia
 - ↳ Pide: descripción, tipo (WEB/BD/RED/SERVIDOR), prioridad (BAJA/MEDIA/ALTA)
 - ↳ Genera ID: obtener_siguiete_id() → max(IDs) + 1
 - ↳ Construye diccionario con todos los campos
 - ↳ Fecha/hora: datetime.now().strftime("%Y-%m-%d") / ("%H:%M:%S")
 - ↳ Añade fila al CSV con DictWriter en modo append
2. Ver Incidencias
 - ↳ Lee CSV y muestra: ID | Fecha | Usuario | Estado | Descripción
3. Salir → break

Descripción de funciones principales

verificar_login(usuario, password) — Lee el CSV de usuarios con DictReader, recorre cada fila y compara usuario y contraseña aplicando strip() para eliminar espacios. Devuelve True si las credenciales son válidas y False en caso contrario.

obtener_siguiente_id()— Abre el CSV de incidencias, extrae todos los valores del campo ID con una lista de comprensión y devuelve $\max(\text{IDs}) + 1$. Si el archivo no existe, devuelve 1 para comenzar desde el principio.

crear_incidencia(usuario_actual)— Solicita los datos al usuario, construye un diccionario con todos los campos (incluyendo la fecha y hora actuales), verifica si el archivo existe para escribir las cabeceras si es la primera incidencia, y añade la nueva fila en modo append usando DictWriter.

listar_incidencias() — Lee todas las filas del CSV con DictReader y las imprime con formato ID | Fecha | Usuario | Estado | Descripción. Si el archivo no existe, muestra el mensaje "No hay incidencias registradas".

menu_principal(usuario)— Bucle while True que muestra el menú y ejecuta la función correspondiente según la opción elegida. La opción 3 rompe el bucle con break.

main() — Función de entrada: solicita credenciales, llama a verificar_login() y, si el login es correcto, llama a menu_principal(). Se ejecuta solo cuando el script se llama directamente gracias a `if __name__ == "__main__":`.

10.2. Panel de Monitorización RustyX

Descripción y contexto

El Panel de Monitorización es una herramienta desarrollada para supervisar en tiempo real el estado de la infraestructura Docker de RustyX (balanceador de carga, dos servidores web y base de datos). Antes de su desarrollo, la supervisión se realizaba manualmente mediante SSH a cada contenedor, lo que implicaba tiempos de detección de fallos de entre 10 y 30 minutos y consumía una parte significativa del tiempo diario de los técnicos.

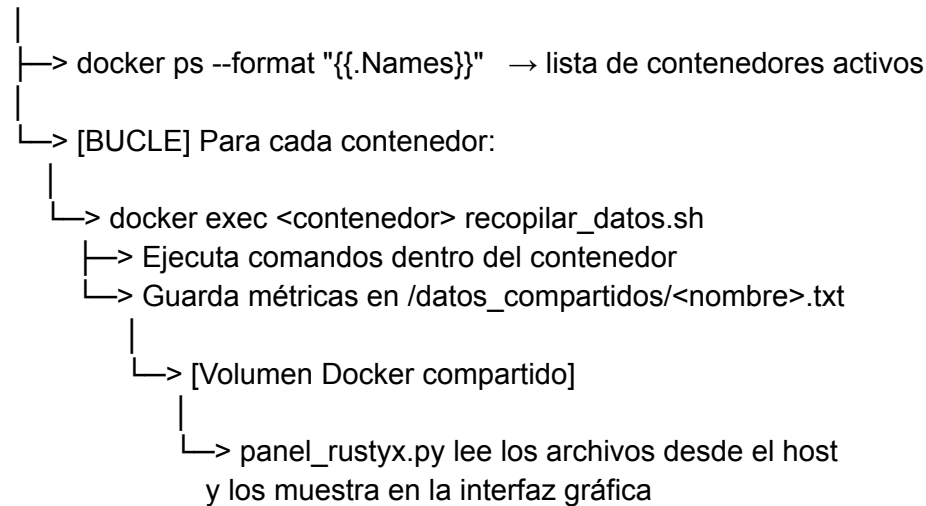
La herramienta se compone de dos partes que trabajan de forma coordinada:

- Script Bash (recopilar_datos.sh): se ejecuta dentro de cada contenedor Docker y recopila las métricas del sistema.
- Interfaz Python (panel_rustyx.py): se ejecuta en el host físico, lee los datos a través de volúmenes Docker compartidos y los presenta en una interfaz gráfica unificada.

Arquitectura del panel

La comunicación entre el script Bash y la interfaz Python se realiza a través de volúmenes Docker compartidos, aprovechando una de las funcionalidades principales de Docker:

HOST FÍSICO



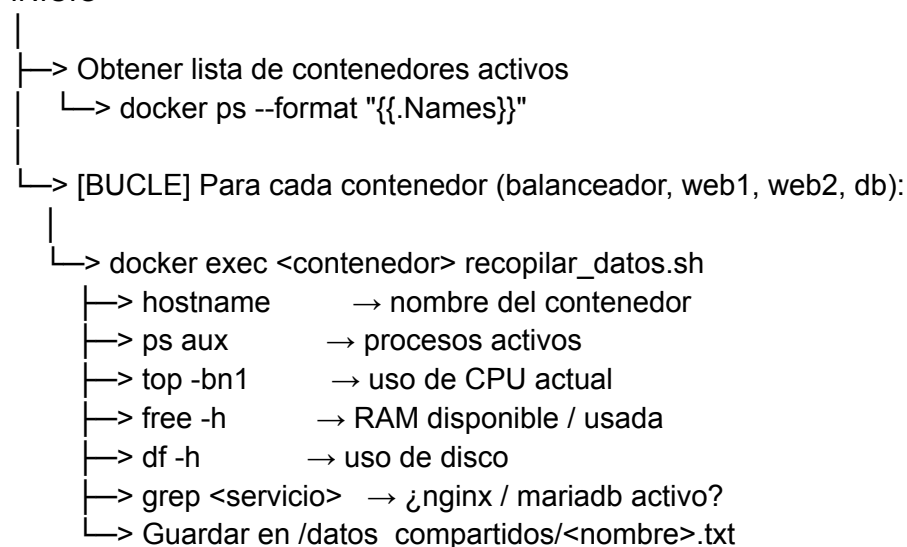
Script Bash — recopilar_datos.sh

Este script se ejecuta dentro de cada contenedor y recopila las métricas del sistema usando herramientas estándar de Linux:

Herramienta	Métrica recopilada	Justificación
hostname	Nombre del contenedor	Identificador único del nodo
ps aux	Lista de procesos con CPU/RAM	Columnas estructuradas para parseo
top -bn1	Snapshot de uso de CPU	Captura puntual sin interactividad
free -h	Uso de memoria RAM	Salida legible en MB/GB
df -h	Uso de disco por filesystem	Estado del almacenamiento
grep <servicio>	Verificación de procesos críticos	Detecta si nginx/mariadb está activo
date '+%Y-%m-%d %H:%M:%S'	Marca de tiempo	Fecha estandarizada y coherente

Flujo del script Bash:

INICIO



FIN

Interfaz gráfica Python — panel_rustyx.py

La interfaz Python se ejecuta en el equipo físico del técnico, lee los archivos del volumen compartido Docker y los presenta de forma visual. Proporciona una visión centralizada de toda la infraestructura de un solo vistazo, eliminando la necesidad de conectarse por SSH a cada contenedor individualmente.

Funcionalidades principales:

Funcionalidad	Descripción
Centralización de infraestructura	Muestra todos los contenedores en una sola pantalla
Estado de contenedores	Indica si cada contenedor está activo o caído
Métricas CPU / RAM / Disco	Valores actualizados por contenedor
Verificación de procesos críticos	Comprueba si nginx, mariadb y php-fpm están en ejecución
Detección automática de caídas	Alerta cuando un servicio deja de responder
Cálculo de uptime	Tiempo de funcionamiento ininterrumpido de cada contenedor
Reducción de trabajo técnico	Elimina la supervisión manual por SSH

Mejoras propuestas para el panel

- Implementar alertas automáticas por correo o mensajería instantánea al detectar una caída.
- Añadir visualización de métricas históricas mediante gráficas temporales.

11. Seguridad

11.1. Principios de Seguridad Aplicados

- Mínimo privilegio: cada componente y usuario tiene acceso únicamente a los recursos estrictamente necesarios para su función.
- Defensa en profundidad: múltiples capas de seguridad (firewall, segmentación de red, GPOs, control de acceso en BD).
- Segmentación de red: separación estricta entre DMZ, red backend y red de datos.
- Centralización del control: Active Directory como único punto de gestión de usuarios, permisos y políticas.

11.2. Seguridad en la Capa de Red

Las reglas de firewall iptables implementan una política de "denegar por defecto, permitir explícitamente". Solo se abren los puertos estrictamente necesarios en cada segmento de red ([ver sección 5.4](#)).

11.3. Seguridad en el Dominio Windows

Las 8 GPOs implementadas cubren los siguientes vectores

Vector de seguridad	GPO
Acceso al sistema (CMD, Panel de Control)	GPO 1
Imagen corporativa y política de contraseñas	GPO 2
Control de horario de acceso	GPO 3
Navegación web no autorizada	GPO 4
Exfiltración de datos por USB	GPO 5
Instalación de software no autorizado	GPO 6
Evasión de restricciones (Administrador de Tareas)	GPO 7
Ejecución de comandos avanzados (Ejecutar)	GPO 8

11.4. Seguridad en la Capa Web

- SSH sin acceso root directo (uso obligatorio de sudo).
- Aplicación web ejecutada bajo el usuario www-data, sin privilegios de sistema.

11.5. Seguridad en la Base de Datos

- Usuario de aplicación rustyx_user con permisos CRUD restringidos a rustyx_db.
- Conexiones remotas limitadas a 192.168.20.0/24 a nivel de configuración de MariaDB.
- Puerto 3306 bloqueado en firewall para cualquier origen fuera de la red backend.
- Sin acceso root remoto a MariaDB.

12. Alta Disponibilidad y Tolerancia a Fallos

12.1. Puntos Críticos del Sistema

Componente	Impacto de fallo	Criticidad
Balanceador de carga	La web queda completamente inaccesible desde internet	CRÍTICA
Base de datos MariaDB	La aplicación no puede servir datos y deja de funcionar	CRÍTICA
Servidor Web Backend 1 o 2	Capacidad reducida al 50%; el servicio continúa activo	MEDIA
Windows Server (AD)	Los usuarios del dominio no pueden autenticarse	ALTA

12.2. Mecanismos de Redundancia Implementados

- Redundancia de servidores web: los dos backends idénticos garantizan que el fallo de uno no interrumpa el servicio, solo reduce la capacidad al 50%.
- Health checks automáticos: Nginx detecta fallos pasivos (errores 5xx/timeout) y redirige el tráfico al nodo disponible sin intervención manual.
- Separación de capas: la arquitectura de tres redes aisladas contiene el impacto de cualquier compromiso en una capa.

12.3. Escenarios de Fallo y Respuesta

Escenario	Respuesta del sistema
Backend 1 cae	El balanceador redirige todo el tráfico a Backend 2 automáticamente
Balanceador cae	Acceso temporal directo al backend (solo emergencia, crea brecha de seguridad)
MariaDB cae	La aplicación devuelve errores hasta que se restaure la base de datos

Nota: el acceso directo al backend como contingencia ante la caída del balanceador se considera únicamente para situaciones estrictamente necesarias, ya que expone los servidores web directamente a internet creando una brecha de seguridad en la arquitectura.

12.4. Propuestas de Mejora de Alta Disponibilidad

- Implementar un segundo balanceador con Keepalived y VIP flotante para eliminar el SPOF del balanceador.

- Configurar un clúster MariaDB (Galera Cluster) para redundancia de la base de datos.
- Implementar NFS para sincronización automática de /var/www/rustyx/ entre los backends.
- Configurar un segundo controlador de dominio (DC secundario) en Active Directory.

13. Resolución de Problemas

13.1. Problemas Habituales

Síntoma	Causa probable	Diagnóstico / Solución
502 Bad Gateway	PHP-FPM caído o socket perdido	systemctl status php8.1-fpm → systemctl restart php8.1-fpm
504 Gateway Timeout	Backend saturado o query SQL pesada	Revisar logs de nginx y mariadb; optimizar consultas
DB Connect Fail	iptables bloquea el puerto 3306	telnet 192.168.30.10 3306; revisar reglas iptables en el host de MariaDB
Usuario no puede hacer login (AD)	Contraseña expirada o cuenta bloqueada	Revisar en ADUC; desbloquear cuenta o restablecer contraseña
GPO no se aplica	El equipo no ha actualizado la política	gpupdate /force en el equipo afectado
DHCP no asigna IP	Ámbito agotado o servicio detenido	Revisar el ámbito en el servidor DHCP; reiniciar el servicio

13.2. Comandos de Diagnóstico Básico

```

bash
# Estado de servicios web
systemctl status nginx php8.1-fpm mariadb

# Conectividad entre capas
ping 192.168.20.10
telnet 192.168.30.10 3306

# Logs principales
tail -f /var/log/nginx/error.log
tail -f /var/log/mysql/error.log

# Contenedores Docker activos
docker ps --format "{{.Names}}"
# Forzar actualización de GPOs (Windows)
gpupdate /force

```

14. Glosario de Términos

Active Directory (AD) — Servicio de directorio de Microsoft que centraliza la autenticación, autorización y gestión de usuarios, equipos y políticas en un dominio Windows.

Backend — Servidores que procesan peticiones detrás de un balanceador o proxy inverso.

Balanceador de carga — Software o hardware que distribuye peticiones entrantes entre múltiples servidores para optimizar recursos y garantizar disponibilidad.

CRUD — Create, Read, Update, Delete. Operaciones básicas de gestión de datos en una base de datos.

DHCP — Dynamic Host Configuration Protocol. Protocolo que asigna automáticamente configuración de red (IP, máscara, gateway, DNS) a los dispositivos de una red.

DMZ — Zona desmilitarizada. Segmento de red intermedio entre internet y la red interna, que aloja servicios accesibles desde el exterior.

DNS — Domain Name System. Sistema que traduce nombres de dominio legibles a direcciones IP.

GPO — Group Policy Object. Objeto de directiva de grupo en Active Directory que permite aplicar configuraciones de seguridad y sistema a usuarios y equipos del dominio.

Health Check — Verificación periódica del estado de un servicio para determinar si está operativo.

MariaDB — Sistema de gestión de bases de datos relacional, fork de MySQL, compatible con el estándar SQL.

Nginx — Servidor web y proxy inverso de alto rendimiento, usado en este proyecto como balanceador de carga y servidor de aplicaciones.

OU (Organizational Unit) — Unidad organizativa en Active Directory. Contenedor que agrupa usuarios, equipos y grupos para aplicar políticas y delegar administración.

PHP-FPM — PHP FastCGI Process Manager. Implementación de FastCGI para PHP que mejora el rendimiento en entornos de alta carga.

Proxy inverso — Servidor que recibe peticiones de clientes y las reenvía a servidores backend, ocultando la infraestructura interna.

Round-Robin — Algoritmo de balanceo de carga que distribuye las peticiones de forma equitativa y cíclica entre los servidores disponibles.

SPOF (Single Point of Failure) — Punto único de fallo. Componente cuya caída provoca la interrupción total del sistema.

Upstream — Conjunto de servidores backend que procesan peticiones, definido en la configuración del balanceador Nginx.

15. Conclusiones y Propuestas de Mejora

15.1. Conclusiones

Se ha diseñado e implementado una infraestructura completa para la empresa RustyX, abarcando desde la base de datos hasta los servicios de red y las herramientas de gestión.

Los principales logros del proyecto son:

- Se ha desplegado un dominio Active Directory completamente funcional con la estructura de OUs, grupos, usuarios, perfiles móviles y 8 GPOs de seguridad orientadas a las necesidades reales de una empresa tecnológica.
- Se ha implementado una arquitectura web LEMP de tres capas sobre Docker con balanceo de carga Round-Robin, redundancia de servidores backend y aislamiento estricto de la base de datos, garantizando la continuidad del servicio ante fallos individuales de componentes.
- Se ha diseñado un modelo de base de datos relacional completo para el catálogo online de videojuegos de RustyX, con todas sus entidades, relaciones e integridad referencial.
- Se han desarrollado dos herramientas de gestión propias: el sistema de tickets RustyX Tickets y el Panel de Monitorización, que reducen significativamente el tiempo de detección y resolución de incidencias.

15.2. Propuestas de Mejora

- Alta disponibilidad del balanceador: implementar Keepalived con VIP flotante para eliminar el SPOF del balanceador de carga.
- Clúster de base de datos: configurar Galera Cluster para MariaDB, eliminando el SPOF de la capa de datos.
- Sincronización automática de contenido web: implementar NFS para replicar /var/www/rustyx/ entre los backends en tiempo real.
- Alertas automáticas: integrar notificaciones por correo o mensajería al detectar caídas en el panel de monitorización.
- Cifrado de contraseñas en RustyX Tickets: sustituir el almacenamiento en texto plano en usuarios.csv por hash bcrypt o equivalente.
- Aplicación web del catálogo: desarrollar el frontend y backend de la aplicación RustyX que consuma la base de datos diseñada.

16. Bibliografía

- Documentación oficial de Microsoft Active Directory — <https://docs.microsoft.com/es-es/windows-server/identity/ad-ds/>
- Documentación oficial de Nginx — <https://nginx.org/en/docs/>
- Documentación oficial de MariaDB — <https://mariadb.com/kb/en/>
- Documentación oficial de Docker — <https://docs.docker.com/>
- Documentación oficial de Certbot / Let's Encrypt — <https://certbot.eff.org/docs/>
- Documentación oficial de Python — <https://docs.python.org/3/>
- Documentación oficial de PowerShell / Active Directory module — <https://docs.microsoft.com/es-es/powershell/module/activedirectory/>
- Claude — <https://claude.ai/>

Parte extendida (Prácticas)

1. Documentación Técnica

1.1. Descripción General

RustyX es una aplicación web desarrollada en PHP que permite gestionar y consultar un catálogo de videojuegos. La página ofrece funcionalidades públicas, como consultar juegos, filtrar resultados, ver detalles de cada videojuego y reproducir tráileres de YouTube embebidos. Además, incluye funcionalidades privadas para usuarios registrados, como valorar juegos, comentar y crear listas personales.

También cuenta con un panel de administración desde el que se pueden gestionar videojuegos, usuarios, comentarios, valoraciones y exportar información en formato CSV.

1.2. Estructura General del Proyecto

El proyecto está organizado en varios archivos PHP, hojas de estilo CSS y recursos estáticos. Los archivos principales son:

- index.php: página principal del catálogo de videojuegos.
- juego.php: página de detalle de un videojuego concreto.
- admin/index.php: panel de administración de videojuegos.
- admin/dashboard.php: panel resumen del administrador.
- admin/exportar.php: exportación de datos en formato CSV.
- config.php: configuración general, conexión a base de datos y funciones auxiliares.
- css/style.css: estilos visuales de la aplicación.
- partials/header.php y partials/footer.php: partes comunes de la interfaz.

1.3. Conexión con la Base de Datos

La aplicación utiliza una base de datos relacional para almacenar la información de usuarios, videojuegos, géneros, plataformas, comentarios, valoraciones y listas.

La conexión se realiza mediante PDO, lo que permite preparar consultas SQL de forma segura. En las consultas que reciben datos del usuario se utilizan parámetros preparados, evitando inyecciones SQL.

Ejemplo de uso:

```
$stmt = $pdo->prepare("SELECT * FROM videojuegos WHERE id_juego=?");  
$stmt->execute([$id]);
```

De esta forma, el valor recibido no se concatena directamente en la consulta SQL.

1.4. Catálogo de Videojuegos

La página principal del catálogo se encuentra en index.php. En ella se cargan los videojuegos desde la base de datos y se muestran en tarjetas.

Cada juego incluye información como:

- Título.
- Desarrollador.
- Precio.
- Imagen de portada.
- Estado del juego.
- Puntuación media.

La consulta principal utiliza varias tablas relacionadas, como videojuegos, generos, plataformas, valoraciones y tablas intermedias para las relaciones muchos a muchos.

1.5. Sistema de Filtros

El catálogo permite filtrar los juegos por distintos criterios:

- Texto de búsqueda.
- Género.
- Plataforma.
- Estado.
- Precio máximo.
- Orden de visualización.

Estos filtros se envían mediante el método GET, por lo que aparecen en la URL. Esto permite mantener los filtros activos al cambiar de página o compartir el enlace.

El filtro de búsqueda utiliza LIKE para buscar coincidencias en el título del juego o en el nombre del desarrollador.

```
$where[] = "(v.titulo LIKE ? OR v.desarrollador LIKE ?)";
```

```
$like = '%'.$filtroBuscar.'%';
```

```
$params[] = $like;
```

```
$params[] = $like;
```

1.6. Paginación de Juegos

La paginación se realiza en tres pasos principales.

Primero, se cuentan todos los juegos que coinciden con los filtros actuales:

```
SELECT COUNT(DISTINCT v.id_juego)
```

Se utiliza DISTINCT porque un mismo juego puede tener varios géneros o plataformas, y sin esta instrucción podría contarse más de una vez.

Después, se calcula el número total de páginas dividiendo el total de resultados entre el número de juegos por página:

```
$totalPags = max(1,(int)ceil($total/$porPagina));
```

Finalmente, se calcula el desplazamiento con OFFSET:

```
$offset = ($pagina-1)*$porPagina;
```

La consulta final usa LIMIT y OFFSET:

```
LIMIT $porPagina OFFSET $offset
```

Esto permite cargar únicamente los juegos de la página actual, evitando traer todos los registros de la base de datos.

Además, los enlaces de paginación mantienen los filtros activos. Para ello se reconstruye la URL conservando los parámetros actuales, excepto pagina, que se sustituye por el número correspondiente.

1.7. Cambio de Formato de los Juegos

El catálogo permite cambiar entre vista de cuadrícula y vista de lista.

Este cambio no modifica los datos ni vuelve a consultar la base de datos. Simplemente cambia la clase CSS del contenedor de juegos:

```
cont.className = 'games-list-view';
```

o

```
cont.className = 'games-grid';
```

La clase games-grid muestra las tarjetas en formato cuadrícula, mientras que games-list-view las organiza verticalmente como una lista.

Además, la preferencia del usuario se guarda en localStorage:

```
localStorage.setItem('rustyx_view', v);
```

Así, cuando el usuario vuelve a entrar en el catálogo, se mantiene la última vista seleccionada.

1.8. Página de Detalle del Juego

La página juego.php muestra la información completa de un videojuego.

Incluye:

- Imagen de portada.
- Título.
- Desarrollador.
- Descripción.
- Precio.
- Fecha de lanzamiento.
- Géneros.
- Plataformas.
- Puntuación media.
- Número de valoraciones.
- Tráiler de YouTube.
- Comentarios.
- Opciones de valoración y listas para usuarios registrados.

El juego se identifica mediante un parámetro id recibido por URL:

```
$id = (int)($_GET['id'] ?? 0);
```

Si no existe un ID válido, se redirige al catálogo.

1.9. Vídeos de YouTube Embebidos

Los tráileres se gestionan a partir de un campo llamado youtube_url. El administrador puede introducir tanto una URL completa como directamente el ID del vídeo.

La función youtubeId() se encarga de extraer el identificador del vídeo:

```
function youtubeId(?string $url): string
```

YouTube identifica cada vídeo con un código de 11 caracteres. Si el valor introducido ya tiene ese formato, se usa directamente. Si es una URL normal, la función extrae el ID mediante una expresión regular.

Después, el vídeo se muestra con un iframe:

```
<iframe src="https://www.youtube-nocookie.com/embed/ID"></iframe>
```

Se usa youtube-nocookie.com, que es la variante de privacidad mejorada de YouTube.

También se utiliza loading="lazy" para que el vídeo no se cargue inmediatamente al abrir la página, sino cuando el usuario se acerca a esa zona. Esto mejora el rendimiento inicial.

En CSS, el contenedor .youtube-embed mantiene la proporción 16:9 usando padding-bottom: 56.25%, lo que permite que el vídeo sea responsive.

1.10. Valoraciones

Los usuarios registrados pueden valorar un juego del 1 al 10.

Cuando un usuario valora un juego, se inserta o actualiza su valoración usando:

ON DUPLICATE KEY UPDATE

Esto evita que un mismo usuario tenga varias valoraciones para el mismo juego. Si ya existía una valoración, se actualiza.

Después de valorar o quitar una valoración, se recalcula la puntuación media del juego.

1.11. Comentarios

Los usuarios registrados pueden comentar en los juegos.

El sistema incluye varias medidas:

- El comentario debe tener una longitud mínima.
- Hay un límite máximo de caracteres.
- Existe un cooldown para evitar comentarios muy seguidos.
- Los usuarios pueden editar o borrar sus comentarios.
- Los administradores pueden moderar comentarios.

Los comentarios se ordenan por fecha, en orden ascendente o descendente según el parámetro recibido.

1.12. Listas Personales

Los usuarios pueden añadir juegos a sus listas personales.

Cada relación entre lista y videojuego puede tener un estado, por ejemplo:

- Pendiente.
- Jugando.
- Completado.

La tabla intermedia lista_videojuego permite relacionar usuarios, listas y juegos sin duplicar información.

1.13. Panel de Administración

El panel de administración permite gestionar los videojuegos del catálogo.

Desde admin/index.php, el administrador puede:

- Crear nuevos juegos.
- Editar juegos existentes.
- Eliminar juegos.
- Subir imágenes de portada.
- Asignar géneros.
- Asignar plataformas.
- Añadir URL o ID de YouTube.
- Cambiar el estado del juego.

Las relaciones entre juegos, géneros y plataformas se actualizan eliminando primero las relaciones anteriores y guardando después las nuevas seleccionadas en los checkboxes.

1.14. Exportación CSV

La exportación se gestiona desde admin/exportar.php.

El administrador puede exportar distintos tipos de datos:

- Valoraciones.
- Usuarios.
- Comentarios.
- Juegos.

El tipo de exportación se recibe por URL:

```
$tipo = $_GET['tipo'] ?? 'valoraciones';
```

Según el tipo indicado, se ejecuta una consulta SQL distinta.

Antes de generar el archivo, se envían cabeceras HTTP para indicar al navegador que debe descargar un CSV:

```
header('Content-Type: text/csv; charset=utf-8');
```

```
header('Content-Disposition: attachment; filename="rustyx_'.$tipo.'_'.date('Ymd_His').'.csv"');
```

El archivo se escribe directamente en la salida del navegador usando:

```
$out = fopen('php://output', 'w');
```

Esto evita crear archivos temporales en el servidor.

También se añade un BOM UTF-8:

```
fprintf($out, chr(0xEF).chr(0xBB).chr(0xBF));
```

Esto ayuda a que Excel interprete correctamente caracteres como tildes y ñes.

Para escribir las filas se usa fputcsv():

```
fputcsv($out, $r);
```

Esta función se encarga automáticamente de separar columnas, escapar comillas y tratar textos que contienen comas o saltos de línea.

1.15. Seguridad y Buenas Prácticas

El proyecto aplica varias prácticas de seguridad:

- Uso de consultas preparadas con PDO.
- Escape de HTML mediante funciones auxiliares como `h()`.
- Control de acceso al panel de administración con `requireAdmin()`.
- Validación de acciones POST.
- Uso de listas blancas para valores como el orden de comentarios.
- Evitar SQL dinámico peligroso en la exportación CSV.
- Validación de tipo y tamaño en la subida de imágenes.

2. Funcionalidad de la Página Web

RustyX es una página web para consultar, valorar y organizar videojuegos.

En la página principal, el usuario puede ver un catálogo de juegos con sus portadas, precios, desarrolladores y puntuaciones. También puede filtrar los resultados por género, plataforma, estado, precio o texto de búsqueda.

El catálogo tiene paginación, por lo que los juegos se muestran por páginas para que la navegación sea más cómoda.

El usuario también puede cambiar la forma de ver los juegos, eligiendo entre una vista en cuadrícula o una vista en lista.

Al entrar en un juego concreto, se muestra una página de detalle con toda su información: descripción, precio, fecha de lanzamiento, géneros, plataformas, puntuación media y tráiler de YouTube si está disponible.

Los usuarios registrados pueden valorar los juegos, escribir comentarios y añadir videojuegos a sus listas personales. Esto les permite llevar un seguimiento de los juegos que quieren jugar, están jugando o ya han completado.

La web también incluye un panel de administración. Desde ahí, los administradores pueden añadir, editar y eliminar videojuegos, gestionar sus imágenes, asignar géneros y plataformas, añadir tráileres de YouTube y exportar datos en formato CSV.

En resumen, RustyX funciona como una plataforma de catálogo y comunidad de videojuegos, combinando consulta pública, participación de usuarios y herramientas de administración.